

面向 AI Agent 轨迹的可配置语义剖析

AgentSight 研究原型

2026 年 7 月 4 日

摘要

AI agent 的执行历史已经不再只是一次 prompt 或一棵 trace tree。一个真实轨迹可能同时包含用户意图、LLM 响应、tool call、API 调用、GUI 动作、浏览器动作、进程、文件、网络、计划步骤、subagent 以及 benchmark 给出的成功、失败、安全、重复和冗余标签。现有 agent observability 工具通常把这些对象固定成 prompt、session 或 span 层级；传统 profiler 则固定成函数或进程层级。这两种做法都会把 profiling 问题过早绑定到某一种边界上，使同一段轨迹很难按任务、阶段、工具、动作、质量标签或人工分组递归折叠。

本文提出一种更小的语义剖析模型：系统只保留两个核心抽象，**operation** 和 **operation stack**。**operation** 是带字段和权重的观测，prompt、session、tool call、process、syscall、GUI action 和 plan step 都只是 operation 的不同形态或字段。**operation stack** 是用户从字段中选择的递归投影，mapping 和 tagging 在 stack 构造前派生字段，而不是创建第三种 profiler 对象。关键点是聚合不是第三个对象，也不是固定 trace tree；聚合由 query time 选择的 stack 形态决定。我们在 Rust agentpprof 中实现该模型，并把论文评估组织成四个核心实验，而不是按 R 编号罗列小实验。E1 验证覆盖和递归折叠：在 15 个轻量采样的公开标注轨迹来源上，agentpprof 把 47,590 个 samples 投影到同一 operation layer；同一批 13,265 个 operations 可以从 9 个 dataset stacks 折叠为 57 个 phase stacks、226 个 semantic stacks、455 个 action stacks，或 3,757 个 fixed-session stacks，而不改变输入 operation。这说明 prompt、session、tool、GUI action、process 和人工边界都只是 operation 字段或 stack 查询，不是新的 profiler 抽象。

E2 是 hidden-label localization/ranking 主实验。在 AgentRewardBench、SATraj-OS、AgentNet 和 OSWorld-Human 的 4 个 oracle-rich 数据集、6 个任务、34,539 个 operations 和 3,699 个 positives 上，我们把 profile groups 当作 ranked localization outputs，比较 flat、fixed-session/span-tree proxy、dataset-native、raw-action 和 operation-stack views。Task-query operation-stack profiling 的 top-5 median work 为 0.0937，而 flat summary 为 1.0；相对 fixed-session drilldown，它在 5/6 tasks 上提高 top-5 recall，并把 median groups 从 285.0 降到 157.5。

E3 隔离机制和 profile-configuration actionability。Rank feature、stack depth、mapping、transfer、diagnostic lens、case packet 和 counterfactual 分析显示：有效配置随 task 和 objective 变化，operation-stack 是 Pareto point，不是 universal default。Executable profile-spec patches 在 5/6 tasks 上改善 AP、top-5 lift 和 first-positive work；supervised held-out boundary-field ablation 把 OSWorld-Human AP 从 0.2402 提到 0.2583，并把 groups 从 108 降到 74，但保留 work 反例。E4 则验证可复现性和 claim integrity：

76 个 tracked profile specs 可确定性 replay，paper gates 检查 source status、数字、guardrails、rubric coverage 和 two-abstraction boundary。

因此本文支持的是 scoped profiling claim：operation-stack profiling 能在公开真实标注 trace 的 dataset-provided hidden labels 下定位、排序和解释任务相关 failure、quality problem 和 semantic boundary，并相对 flat summary 减少 inspection work、相对 fixed-session/span-tree proxy 减少 fragmentation。本文不声称提升 human productivity，不声称自动发现所有 intent boundary，不声称 metric dominance，也不声称完整兼容现有 trace ecosystem。

1 问题

AI agent 轨迹的调试问题正在从单次调用回放变成跨任务剖析问题。开发者不只想知道某个 span 什么时候开始和结束，也想知道哪类任务导致最多重复点击，哪些 GUI 轨迹反复输入，哪个工具调用族对应失败，哪些安全攻击类型集中在某些操作阶段，或者一个 prompt 序列能否被折叠成 task、phase、action 和人工 subtask 等多个深度。如果 profiler 把 session、prompt 或 tool call 写死成核心层级，这些问题会被迫使用同一个边界。如果 profiler 只保留进程或函数栈，上层语义和 benchmark oracle 又会丢失。

本文的出发点是把 agent trace 中所有可剖析对象都降到同一层。一个 prompt 可以是 operation，一个 tool call 可以是 operation，一串 prompt 也可以通过 mapping 被派生为同一个 intent 或 task operation field。同样，GUI action、API call、browser action、process event、syscall、安全标签、step correctness 和人工 grouped-action id 都不是新的 profiler 抽象，而是 operation fields。Profiler 的核心任务不是决定唯一正确边界，而是让用户和实验可以在同一批 operations 上选择 stack 字段、派生 mappings，并得到不同深度的递归折叠。

这个问题有两个系统挑战。第一，抽象必须足够小，否则 prompt、session、tool call、plan、subagent 和 process 会各自形成独立 pipeline。第二，抽象又必须足够可配置，否则 flamegraph 只会成为固定视图，不能回答边界、失败、安全和质量诊断问题。本文的目标不是证明某个单一 flamegraph 最好，而是证明 operation 和 operation stack 足以覆盖多类真实标注 agent 轨迹，并且 stack shape 与 aggregation 的选择会系统性改变质量、边界、定位、work 和碎片化分析结果。

2 设计

2.1 两个核心抽象

operation 是唯一的样本级对象。每个 operation 是一个带权重的字段集合，例如 `op=prompt`、`op=llm`、`op=tool`、`tool=browser`、`action=click`、`phase=navigate`、`status=failure` 或 `step_correct=incorrect`。这些字段可以来自本地 Codex/Claude session，也可以来自外

部 benchmark 的标注轨迹。实现上，外部轨迹使用每行一个 JSON object 的 operation JSONL，并把原始任务文本、截图和长对话默认排除在提交 artifact 之外。

operation stack 是从 operation fields 中选择出的有序 frame 列表。例如同一批 operations 可以用 project,dataset 查看数据来源，也可以用 project,dataset,task,phase,op,tool,action,status 查看动作深度，或者加入 human_group、step_correct、safety 和 looping 做诊断视图。Stack 不是 trace tree，也不是 session hierarchy。它是一个查询结果，用户可以按问题改变深度和字段。

2.2 Mapping 和 tagging

Mapping 和 tagging 是派生 operation fields 的一等机制。它们在 stack 构造前运行，读取 operation 的 key=value 文本，并写入新的字段。例如 desktop action 中的 type 和 key 应先被映射到 phase=input，再考虑通用 web action verb；API/tool traces 应先进入 tool/API phase family，再处理 search 或 create 这类词。这一顺序来自跨 web、desktop、API/tool 和 step-quality 轨迹时暴露出的关键设计约束。如果通用 action rule 抢在 operation-family rule 前面运行，desktop、web 和 API 轨迹会被错误合并。

Mapping 不引入第三个抽象。它只是在 operation 上写字段，使同一个 stack specification 可以跨数据集复用。当前 Rust CLI 支持 inline --op-map FIELD:LABEL=REGEX 和文件形式 --op-map-file.--where FIELD=REGEX 和 FIELD!=REGEX 在 mapping 之后、stack 构造之前选择 operation 子集，多条 predicate 取 AND；它实现形式化模型里的 ϕ ，不是新的 profiler 对象。script/operation_map_infer.py 可以从已标注 operations 中生成同样格式的规则文件，并用这些规则测试 held-out 和 leave-dataset-out 泛化。--profile-spec 把 operation files、mapping files、predicate、view、stack 和输出路径打包成 JSON 配置，便于 reviewer 复现实验；命令行参数仍可覆盖 spec 默认值。--deterministic-output 或 spec 中的 deterministic_output 可以把 JSON generated_at 和 pprof profile time 固定为可复现值，用于 byte-stable artifact checks。它不改变模型本身，只是把已有 operation 和 operation-stack 查询显式记录下来。

2.3 Views 和非火焰图分析

--view 决定采样和权重，例如 operation count、token、file、network 或 time。--stack 决定递归折叠形状。Folded stack 和 SVG flamegraph 是其中一种输出，但不是唯一输出。我们同时生成 JSON profile、stack tree、top-field table、parent-child transition、depth sweep、boundary F1、V-measure、coverage report、grouped-action report 和 HTML quality report。这些视图共享同一 operation 输入和 stack 语法，因此 failure、safety、looping、redundancy 和 human group 可以被当作普通字段评分，而不是另建 failure profiler 或 safety profiler。

3 实现

agentpprof 是当前被测实现。它是 Rust CLI，读取本地 Codex/Claude 历史或外部 operation JSONL，并输出 pprof-compatible profile、folded stacks、SVG 和 JSON。核心命令形态如下：

```
agentpprof --operation-file ops.jsonl --view operations
--stack project,dataset,task,phase,op,tool,
action,status --op-map-file operation-map.txt
--where task=verify
-o out.folded
```

外部数据集转换器位于 script/agent_trace_datasets.py。转换器只下载或流式读取所需公开文件的采样前缀。对于包含截图、长 prompt 或完整对话的数据集，默认保存 redacted rows 和 operation JSONL，不提交原始数据。本地 agent session 通过 agent-session 解析为 agentsight.agent-session.trace.v1 trace；agentpprof 可以用 --export-trace 写出 filesystem-normalized trace，用 --trace-file 读回，也可以通过 script/agent_trace_convert.py to-operations 转成 operation JSONL。script/agent_trace_exchange_eval.py 将这三步、filesystem/tool-command portability check 和 folded-output equality check 固化为一个可复现实验；该检查不声称 prompt/LLM preview 已完整脱敏。agentpprof 还可以用 --export-standard-trace 写出 Chrome/Perfetto-style traceEvents，再用 --standard-trace-file 导入为普通 operations 后折叠。script/agent_trace_convert.py export-standard --format chrome 和 import-standard --format chrome 保留为显式中间文件脚本；script/agent_trace_chrome_exchange_eval.py 验证 session fixture 路径不改变 folded operation-stack 输出。R353 进一步用 tracked R324 real labeled visible-operation JSONL 的 512-row prefix 验证 operation-file 路径：512 operations 导出为 512 个 Chrome complete events，经 --standard-trace-file 导入后仍得到 byte-identical 的 512-sample / 11-stack folded 输出；这仍是 exchange check，不是新的 accuracy result。质量评估脚本位于 script/operation_stack_quality.py，它复用 agentpprof 的 operation mapping contract，并计算字段覆盖率、oracle V-measure 和序列相邻边界 F1。Stack 结构分析脚本位于 script/operation_stack_analysis.py，用于生成 flamegraph 之外的 tree、top-kind 和 transition 报告。

这个实现刻意保留简单接口。它不像 jq 那样完整表达 JSON 查询语言，但它已经把 profiler 的关键自由度暴露为 flags 和可提交配置：数据源由 --trace-file 或 --operation-file 选择，权重由 --view 选择，查询子集由 --where 选择，字段派生由 --op-map 选择，递归边界由 --stack 选择，JSON group 排序由 --rank-rule、--rank-op-rule 和 --rank-mode 选择，复现实验由 --profile-spec 固化。R321 用同一份 R300 真实标注 operation JSONL 验证该实现路径：profile spec 中的 mapping-derived where_rules 分别选出 729/729、714/714 和 4,285/4,285 个预期 operations 后再折叠。R322 进一步让 Rust JSON profile 输出 visible-rule ranked operation-stack groups；在同一 6 个标注任务上，rank_rules 相比 width ranking 的 AP 提升覆盖 4/6 tasks，top-5 lift 提升覆盖 3/6 tasks，同时 SATraj 和 side-effect 反例显示二值 stack-text boost 仍弱于 R320 的 group-feature query-aware ranker。R323 把该反例转化为 rank-mode 机制隔离：rule-score 相比 width-boost 的 AP 提升覆盖 4/6 tasks，top-5 lift 提升覆盖 4/6 tasks，并把 SATraj first-positive work 从 0.6376 降到 0.0842；side-effect 和 OSWorld-Human 仍保留为 depth/ranker counterexamples。R324 把 per-

operation visible feature density 接到 Rust profiler: 脚本先从 R300 source 派生删除 oracle fields 的 visible-operation JSONL, 再让 `rank_op_rules` 匹配 mapping/filtering 后的单个 operation field=value token, 并在 folded group 内聚合 matched operation weight; semantic stack 上 AP 相比 width 提升 5/6 tasks、top-5 lift 提升 4/6 tasks、first-positive work 改善 5/6 tasks, coarse stack 也在 AP 上提升 4/6 tasks, 同时保留 OSWorld-Human 反例。R325 在同一 Rust profile-spec 路径上做 leave-one-feature ablation: 它发现 7 个 critical feature instances、3 个 misleading feature instances, 并显示 coarse stack 只在 2/6 tasks 上 AP 更好但在 6/6 tasks 上减少 group 数。R326 进一步测试 rank-policy robustness: global equal visible feature bank 在 semantic/coarse stack 上 AP 相比 width 分别赢 4/6 和 5/6 tasks, task-equal 在 8/12 task/depth variants 上与 weighted task policy 的 AP 差距不超过 0.02, R325-guided repair 在 2/3 misleading-feature cases 上改善 AP、在 2/3 cases 上改善 first-positive work, 其中 1/3 同时改善两者; repair 是 post-hoc actionability check, 不是 label-free deployment policy。R329 再做 target-held-out transfer: 候选 policy 包括 global equal bank 和 6 个 source-task equal-weight policies, 选择 leave-task 或 leave-dataset policy 时只使用非目标任务的离线评分, 目标 hidden labels 仅用于最终 scoring; semantic/coarse stack 的 AP wins 都是 4/6, semantic first-positive work 在 6/6 tasks 改善, 但 side-effect、SATraj 和 boundary 反例说明这仍不是通用 label-free ranker。R327 再把这些可提交 profile specs 作为 reproducibility/cost workload: R300 的 4 个 view specs、R324 的 12 个 rank-feature specs 和 R326 的 60 个 robustness specs 均由 Rust agentpprof --profile-spec 重复执行两次; semantic profile hash 全部稳定, raw-byte hash 只有 4/76 稳定, 因为 JSON profile 包含生成时间戳。R328 在同一 76 个 specs 上启用 --deterministic-output, 使 raw-byte hash 也达到 76/76 稳定; 这关闭的是 artifact byte-stability 缺口, 不是新的 accuracy benchmark。因此 prompt、session、toolcall、process、syscall、plan 和 sub-agent 都可以在同一套 mechanism 中出现, 而不是成为互不兼容的 object model。

4 实验方法

评估遵循八个原则。第一, 只使用公开的、别人已经标注好的 agent 轨迹或交互轨迹。我们使用 Hugging Face Dataset Viewer、HF repo JSONL、公开 GitHub JSON 和公开 benchmark artifacts 进行轻量采样, 不同步完整测试集, 也不提交原始外部样本。第二, 每个 claim 都必须对应可执行 oracle。对于 phase/action 结构使用 V-measure 和相邻边界 F1; 对于 grouped-action 使用人工 group id; 对于 looping、safety、step correctness 和 redundancy 使用数据集原生标签; 对于递归深度使用同一批 operations 下 unique stack 和 compression 变化。第三, 负结果保留在论文中。例如 AgentNet 的 task status 很弱地预测 per-step correctness, repeat signal 也很弱地预测 step redundancy。这说明 profiler 能承载这些诊断字段, 但简单 mapping 不能替代专门质量模型。第四, 论文 claim 必须从 artifact 机械回溯。Claim synthesis、reviewer evidence packet、paper value/novelty synthesis、paper evidence matrix 和 submission audit 都只读取 tracked/clean result files, 并输出 claim verdict、paper-ready wording、must-not-claim 和 source

paths。这些文件不是新实验, 也不是 profiler object。它们的作用是防止摘要、结果、局限和结论在数字、scope 或 unsupported claim 上漂移。第五, boundary backend 必须仍然服从两个抽象。监督式 adjacent-boundary backend 只写回 learned_group_pattern、learned_segment_pattern、learned_segment_position 和 learned_boundary_prev 等 operation fields。递归折叠仍由 agentpprof 使用普通 --profile-spec 和 --stack 完成。每个候选标签家族还必须先通过 suitability 和 simple-baseline gate, 防止把 safety、history context 或 repetition proxy 错当作语义边界。第六, 真实问题价值必须先通过可复现 proxy 任务收敛。我们从 AgentRewardBench、SATraj-OS、AgentNet 和 OSWorld-Human 的已有 operation JSONL 构造 6 个 oracle-backed analysis tasks, 比较 flat、fixed-session、semantic operation-stack 和 label-drilldown 四种 stack specs。指标包括 positive lift、覆盖 50% positives 的 inspection fraction、group count、top-group session support、selected recall 和 selected work。这仍不是人类用户实验, 但它把“profiler 是否解决真实问题”从叙述变成可审计的 proxy result。第七, 默认浏览、ranking 和 case evidence 都必须避免答案泄漏。Visible packets 不包含 oracle-positive 字段, hidden answer key 单独保存 positives。非 oracle rankers 只读 status、repeat_signal、phase、action 和 environment 等可见字段, 不读 looping、safety、step_correct 或 target_positive。因此 first-positive、first-enriched 和 high-lift evidence 只是对同一 operation/operation-stack packet 的隐藏标签回放评分, 而不是 automatic detection 或 human utility。第八, R320 把 profiler 输出直接当作 ranking/localization result。它在同一 6 个真实标注任务上比较 flat、fixed-session、dataset-native hierarchy、raw action stack、operation-stack、label-drilldown oracle view, 以及 width、visible-risk、query-aware 和 oracle-upper-bound rankers。非 oracle policies 只使用可见 operation fields, 隐藏标签只用于 scoring。指标包括 precision@k、recall@operation-budget、F1@k、AUPRC-style average precision、nDCG、work-to-first-positive、groups-to-50% recall、group count 和 task-level profile-configuration insight。这个设置把 profiling paper 的主 claim 从“analyst 是否更快”改为“profiler 自身是否能在真实 workload 上相对 dataset-provided hidden labels 定位、排序并指导优化”。

4.1 Hierarchy ablation baselines

本文比较的是 hierarchy choices, 而不是把每个已有 observability 产品完整复刻一遍。Flat profile 把每个任务压成一个 summary packet, 用来检验 operation stack 是否比无层级摘要更 selective。Fixed-session drilldown 把 session 或 demo id 放进 stack, 用来模拟 session/prompt-first 调试界面和传统 trace-tree drilldown 的碎片化风险。Prompt/session tree、tool-call hierarchy 和 OTel/span-like trace tree 在本实验中都被建模为不同 stack specs: 它们仍然读取同一批 operations, 只是把 session、prompt、tool、process 或 trace id 放在更靠前的 frames。Label-drilldown 是 oracle-aware upper-bound style view, 它把 hidden label 直接放入 stack, 用来界定“如果答案字段已经可见”时的上限, 而不作为真实用户界面 baseline。这些 ablations 的目的, 是隔离边界选择对聚合、selectivity、recall 和 inspected work 的影响。因此本文不会声称 agentpprof 支配所有生产 observability UI, 只声称 operation stack 把这些边界选择统一成可配置查询。

表 1: 公开标注轨迹数据源和当前用途。ScaleCUA 是补充点, 主结论主要依赖前 14 个 oracle-rich 数据源。

数据源	原生标注	转换方式	主要用途
WebLINX, Mind2Web, AgentTrek, WebShop API-Bank, ToolBench, tau-bench	web action, demo/task, reward 或 action history API/tool call, solution path, 多轮 user/assistant/tool messages, outcome	HF Dataset Viewer 或 HF repo shard HF rows 或 repo JSONL	web navigation 和 shopping trajectories 的跨源 smoke。 tool/API/dialogue phases 和 user-agent-tool operation 形态。
SWE-agent	issue trajectory, command action, success target	HF Dataset Viewer	software-agent command 轨迹。
AndroidControl, GUI-Odyssey, AgentRewardBench	mobile/cross-app step action 与 instruction expert success, side-effect, looping, optimality	public/HF samples annotation rows 加 cleaned BrowserGym steps	移动 GUI action depth 和跨 app scale。 failure, looping, side-effect 的非 flame-graph 诊断。
SATraj-OS	desktop action, success, safety, reward, attack type	HF safety config	桌面安全和 attack-type operation fields。
OSWorld-Human	human single-action 和 grouped-action 轨迹	GitHub JSONL files	人工 grouped-action 边界 oracle。
AgentNet	human desktop PyAutoGUI, task outcome, step correctness, redundancy	HF repo JSONL streaming	最大 human desktop step-quality oracle。
ScaleCUA	GUI navigation action, previous-operation context, gold action	HF annotation JSONL streaming	补充多步 GUI history-depth smoke。

5 结果

本文的结果不应按 R 编号或零散 probe 展开。论文主体收敛为四个核心实验: E1 验证 operation coverage、递归 stack folding、field derivation 和人工边界都能落在同一个两抽象模型中; E2 是 hidden-label localization/ranking 主实验; E3 通过 ranker、stack、mapping、transfer、case 和 profile-spec patch 消融隔离机制与 actionability; E4 检查 profile-spec 可复现性、离线成本和 claim-integrity guardrails。R 编号只作为 provenance, 不是论文的小节结构。

5.1 E1: 覆盖、递归折叠和边界字段

R361 将本节的 evidence contract 固定为: claim 是两抽象模型能覆盖异构 agent traces 且不把 prompt/session/tool 边界写死; oracle 是 public dataset labels, native trajectory fields, OSWorld-Human human boundary 和 AgentNet quality labels; baseline 包括 dataset-native stacks, no-map folding, fixed-session stacks, profile-spec replay 和 trace round trip; metric 包括 operation coverage, unique-stack range, mapping compression, boundary F1/V-measure 和 replay equality; counterpoint 是这只能支持 configurable operation stack folding, 不能支持完整生态兼容或自动恢复所有 latent intent。Prompt, session, tool, process 和 syscall 都只能作为 operation forms 或 operation fields 出现, not new profiler objects。RQ1 问的是: 同一 operation layer 能否只通过改变 mappings 和 stack fields, 被递归折叠成 task, phase, action, boundary 和 fixed-session 等多种 stack, 而不改变输入记录、不引入第三个 profiler 抽象。

核心 14 数据集产生 42,590 个 operations 和 1,497 个 unique stacks, compression ratio 为 28.45。加入 ScaleCUA 补充样本后, 总量为 47,590 个 operations 和 1,611 个 unique stacks, compression ratio 为 29.541。这些 operations 覆盖 web、mobile、desktop、software、API、tool-agent-user dialogue, expert failure labels, safety labels, human grouped-action labels 和 desktop step-quality labels。所有这些标签都以 operation fields 形式进入同一 pipeline。没有任何数据集需要新增 prompt, session, GUI, safety 或 quality-specific profiler object。

这支持 C1 的机制性版本: agentpprof 可以把异构 agent 轨迹统一为 operations, 并用用户指定 stack 折叠。它不支持所有 agent 数据都容易转换。适合该方法的轨迹通

常具备四个特征: 有有序 step 或 turn, 有稳定 session/task id, 有动作或阶段标签, 有 outcome, quality, group 或 safety 等较高层 oracle。只有静态截图 grounding、缺少顺序、缺少 action label、强依赖大图片归档或 gated access 的数据源更适合作为后续扩展, 而不是当前主证据。

递归 stack-depth sweep. R286 在同一批 13,265 个 operations 上扫过 8 个 stack depths。只保留 dataset 时有 9 个 stacks; 加入 task 后为 11 个; phase depth 为 57 个; tool/semantic depth 为 226 个; action depth 为 455 个; 加入固定 session 后膨胀到 3,757 个。相对于 dataset depth, 固定 session 的 unique stacks 增加 417.4 倍。这个结果直接反驳把 session 或 prompt 作为默认 profiling 边界的设计。Session 是有用 drilldown field, 但如果默认写死在 stack 里, 它会把可聚合的行为切碎。

早期 3,797-operation set 上也出现同样信号。Flat/mapped stack 为 103 个 unique stacks, 而固定 demo/session stack 为 1,083 个 unique stacks。这说明固定边界会带来约 10.5 倍碎片化, 而 mapped operation stack 可以保留聚合同时增加语义深度。因此论文中的 novelty 不是“画 flamegraph”, 而是把 agent profiling 的边界选择变成 operation-stack query。

Profile spec 把这个 query 自由度变成可复现实验配置。同一个 AgentNet spec 引用相同 operation JSONL 和 op-map 文件, 默认产生 608 个 diagnostic stacks; 命令行只覆盖 --stack 和输出路径后, 样本数保持 16,741 不变, unique stacks 降到 83。这说明 stack 深度是可审计、可覆盖的查询参数, 而不是写死在 prompt/session 或某个可视化中的层级。

这种可配置性不只改变图的形状, 也改变 reviewer 能验证的问题。同一套 outputs 同时给出 depth sweep, stack tree, transition/top-field, quality report, grouped-boundary report 和 history-depth report。这些报告回答同一个机制问题: 一个 profiler 是否可以把边界选择延后到查询时, 而不是在采集时固定为 prompt, session 或 span tree。当前证据支持三条 scoped 结论。第一, 异构轨迹和本地 session 可以先进入同一 operation layer, 再用用户选择的 stack 折叠。第二, recursive stacks 能表达 dataset, task, phase, tool, action, human-group, safety 和 quality-label 等多种深度, 但不能声称恢复所有真实意图边界。第三, field derivation 是可扩展接口, 因为 mapping 和 su-

pervised backend 只写入 operation fields, profiler 仍按 operation stack 折叠。这些机制结果给出 paper novelty, 但 user-utility、通用 boundary detector 和完整 trace-platform compatibility 仍是后续 gate。

Field derivation 和 boundary probes. 本节支持的是 configurable deterministic/supervised field derivation 在合适标签家族上的迁移, 而不是通用无监督边界发现。

R281 从 13,265 个 labeled operations 中生成 10 条 mapping rules, 复现手写 mapping 的 folded 输出。R282 使用 dataset-stratified held-out split, 在 9,275 个 train operations 上生成规则, 并在 3,990 个 held-out operations 上评估。Mapped stack 产生 209 个 held-out stacks, compression ratio 为 19.091; 无 mapping baseline 为 284 个 stacks, compression ratio 为 14.049。Task/dataset V-measure 从 0.837 提升到 0.853。Phase/action boundary F1 为 0.777, precision 为 1.0, 说明当前 mapping 是保守的。它不会制造 false positive phase changes, 但会合并一些 fine-grained action changes。

R283-R285 进一步做 leave-dataset-out。在修正 API/tool phase precedence 后, R285 对 9 个 held-out datasets 中的 6 个减少 unique stacks, 0 个出现 negative stack reduction, weighted stack reduction 为每 1,000 operations 1162.005。这说明 mapping 不是单个数据集的 ad hoc pretty printer。它仍然是 deterministic taxonomy-seeded mapping, 不是无监督边界发现。R297 开始测试更强的 boundary backend。它在 OSWorld-Human exact-aligned sessions 上按 session 做 deterministic split: 191 个 train sessions 产生 2,655 个 train adjacent pairs, 96 个 test sessions 产生 1,036 个 test adjacent pairs。模型使用非 oracle operation fields 训练 Bernoulli adjacent-boundary predictor, 明确排除 human_group、group_index、group_position、group_size、group_pattern 和 group_alignment。在 held-out pairs 上, learned backend 的 precision 为 0.7400, recall 为 0.8102, F1 为 0.7735; phase-change baseline 为 0.2919, action-change 为 0.5142, conservative group_pattern reference 为 0.5810。随后脚本把预测边界写成 learned_group_pattern 和 learned_group_position 字段, Rust agentpprof 在 1,132 个 held-out operations 上折叠得到 74 个 stacks。当前可支持的 claim 是 configurable deterministic/supervised field derivation 能在 held-out 设置下改善语义聚合; 完整 model-backed 或 unsupervised boundary detector 仍是后续工作。

R299 将这个 backend 接口复制到现有标注家族, 并加入 suitability/calibration gate。脚本检查 OSWorld-Human human-group、AgentNet step correctness、AgentNet step redundancy、AgentRewardBench looping、SATraj safety、ScaleCUA history state 和 tau-bench tool-dialogue role 共 7 个候选。其中 SATraj safety 在当前样本中没有 session-local adjacent positives, ScaleCUA history state 是 context marker 而不是语义边界, tau-bench 在 R287 没有 tracked operation JSONL, 因此不训练。四个可训练候选的 held-out learned F1 分别是 OSWorld-Human 0.6916、AgentNet step-correct 0.3197、AgentNet step-redundant 0.3361 和 AgentRewardBench looping 0.7833。AgentNet 两个质量状态边界都优于 always-boundary baseline (0.2155 和 0.2645), 但 precision 低且 calibration error 仍明显; AgentRewardBench looping 虽然 learned F1 达到 0.7833, 却被

repeat_signal_change baseline 的 1.0 F1 完全解释。这把 C3 的范围收得更清楚: backend 可以统一地派生 stackable operation fields, 但每个标签家族都必须证明它真的是边界 oracle, 并且必须胜过简单 sequence-derived fields。

Human-boundary oracle. OSWorld-Human 是当前最强的 grouped-boundary oracle。R290 转换全部 369 个 human desktop tasks, 得到 6,010 个 single-action operations。转换器把 grouped-action 序列展平后与 single-action 序列对齐, 只有 exact-aligned tasks 才写入 human_group、group_pattern 和 group_position 字段。最终 320 个 tasks、4,011 个 operations 和 2,075 个 session-local groups 进入 grouped-boundary scoring; 31 个 content-mismatch tasks 和 18 个 length-mismatch tasks 保留用于 action profiling, 但不被当作 gold group oracle。

在 OSWorld-Human 上, 普通 action-depth stack 有 482 个 unique stacks, grouped-depth stack 降到 106 个。Phase/action boundary F1 为 0.638, precision 为 1.0。更重要的是, 保守的 group_pattern 对人工 human_group boundary 的 F1 为 0.627, precision 为 1.0, recall 为 0.456。这不是完美边界 detector, 但它给出有意义结论: operation stack 可以在同一条单动作序列上选择 action-depth 或 group-depth, 并以人工 grouped-action 标签作为 oracle 验证。Prompt 或 session 没有参与这个抽象。

5.2 E2: Hidden-label localization 和 ranking

R361 将本节的 evidence contract 固定为: claim 是 operation-stack profiling 能在真实 labeled traces 上 faithful localization 和 ranking; oracle 是 6 个 oracle-backed tasks 中 34,539 个 operations 和 3,699 个 hidden positives; baseline 包括 flat summary、fixed-session/span-tree proxy、dataset-native hierarchy、raw-action stack、operation-stack width、operation-stack query-aware、label-drilldown 和 oracle upper bound; metric 包括 precision@k、recall、F1、AUPRC-style AP、nDCG、recall/F1@work budget、work-to-first-positive 和 group count; counterpoint 是 flat 和 dataset-native 可以赢 broad-recall 或 nDCG 目标, 所以主 claim 是 Pareto tradeoff, 不是 metric dominance。RQ2 问的是: hot stacks 和 top-ranked groups 是否真的对应 hidden positives, 并且相对 flat summary 需要更少 inspection work、相对 fixed-session drilldown/span-tree proxy 产生更少 fragmentation。

AgentRewardBench、SATraj-OS 和 AgentNet 说明 operation stack 不局限于 flamegraph 热点。AgentRewardBench 的 729 个 expert-reviewed web-agent operations 全部携带 success、side-effect、looping 和 optimality 字段。Action-derived repeat_signal 对 expert looping label 的 V-measure 为 0.378, 而单步 step_error 对 looping 的 V-measure 只有 0.011。这说明 sequence-derived operation fields 可以捕捉一部分真实 failure mode, 也说明简单错误标签不能替代序列诊断。

SATraj-OS 的 4,285 个 desktop computer-use operations 全部携带 safety、attack type、status 和 reward-related fields。其中 622 个 operations 标为 unsafe, attack type 覆盖 account、induced-text、popup、OS 和 unknown-file。这些标签与 phase/action 关系较弱, 例如 attack-type/action V-measure 只有 0.093。这个负结果是有价值的: 安全攻击类别不是 action taxonomy 的简单函数, 因此应该作为独

立 operation field 被折叠和过滤, 而不是强行塞进 phase。

AgentNet 是当前最大的 human desktop step-quality oracle。R291 采样 1,000 个 Ubuntu tasks, 得到 16,741 个 PyAutoGUI operations。所有 operations 都有 task outcome、alignment score、efficiency score、difficulty、step correctness 和 redundancy 字段。Step correctness 覆盖 13,844 correct、874 incorrect 和 2,023 unknown; step redundancy 覆盖 9,334 necessary、733 redundant 和 6,674 unknown。Task status 对 step correctness 的 V-measure 只有 0.015, repeat signal 对 step redundancy 的 V-measure 只有 0.010。这说明 profiler 的价值不是用一个粗标签预测所有质量问题, 而是把质量 oracle 保留为可折叠、可 drilldown、可评分的 operation fields。

R300 把这些诊断目标组织成 6 个自动化 real-problem proxy tasks。任务覆盖 AgentRewardBench looping 和 side-effect、SATraj unsafe、AgentNet incorrect/redundant steps, 以及 OSWorld-Human group starts, 总计 34,539 个 operations。在同一 operation JSONL 上, Rust agentpprof 的 flat profile 只有 6 个 stacks, fixed-session profile 有 2,012 个 stacks, semantic operation-stack profile 有 944 个 stacks, label-drilldown profile 有 318 个 stacks。按 oracle-sorted clustering quality 评价, semantic operation stack 相比 flat summary 的 median top-positive lift 为 5.726x, 覆盖 50% positives 的 median inspection fraction 从 1.0 降到 0.2879。相比 fixed-session stack, semantic stack 的 group count ratio 为 0.554, top groups 的 session support ratio 为 5.5, 说明它更适合跨轨迹诊断; 但 inspection fraction ratio 为 1.302, 说明 fixed-session drilldown 在部分 instance-local tasks 上仍更快。这支持的是 inspectability 和 aggregation claim, 不是人类效率 claim。

R301 用更严格的 label-hidden analyst-task proxy 复核 R300。它把 visible packet 和 hidden answer key 分开, 默认只按 group width 排序。在 30% inspected-operation budget 下, operation-stack 的 median positive recall 为 0.336, 检查 4.5 groups; fixed-session 为 0.284, 检查 25.5 groups。在 top-10 width-ranked groups 下, operation-stack 的 median recall 为 0.641, 而 fixed-session 为 0.195。这个结果说明 operation stack 能把跨 session 的问题压缩成更少可检查单元, 但它并不保证默认宽度排序总是最省 operation budget。

R302 进一步比较 label-hidden ranking policies。在 top-10 groups 下, query-aware operation-stack ranking 只检查 0.116 operation fraction, positive lift 为 1.587; width ranking 检查 0.671 operation fraction, lift 为 1.079。在 30% operation budget 下, query-aware ranking 把 operation-stack recall 从 0.340 提高到 0.390, 但检查 groups 从 4.5 增加到 39.5。这说明 profiler 可以支持不同分析 policy 的 precision/recall tradeoff, 但这些 policy 仍是 visible-field heuristics, 不是 learned detector。

R304 将 R302 的 ranking policy 变成 reviewer-facing case packet。每个任务取 top-5 query-aware operation-stack groups, visible packet 不包含 oracle fields, hidden answer key 单独保存 positives。在 30 个 case groups 上, median inspected operation fraction 为 0.0937, median positive recall 为 0.188, median positive lift 为 1.6509。SATraj unsafe cases 最集中, work fraction 为 0.042、recall 为 0.262、lift 为 6.238; AgentNet quality cases 的 recall 很低, 但在很小 work fraction 上仍有 1.98 以上 lift。这个结果适合作为

case-study evidence, 而不是 detector evidence。

R305 对同一 case-packet policy 加入 flat 和 fixed-session baseline。Flat view 只有每个任务一个 packet, 因此 median recall 为 1.0, 但必须检查 1.0 operation fraction。Fixed-session view 的 median work fraction 为 0.0163、recall 为 0.0226、lift 为 1.6615。Operation-stack view 的 median work fraction 为 0.0937、recall 为 0.188、lift 为 1.6509。相对 fixed-session, operation-stack 的 median recall ratio 为 3.63、lift ratio 为 1.268, 但 inspected-work ratio 为 1.717。这支持“operation stack 是 flat 与 fixed-session 之间的可配置分析折中”, 而不是“operation stack 在每个任务上都更省 work”。

R320 把前面的 case-packet proxy 升级为 hidden-label profiler benchmark。它不再只报告 selected recall/lift, 而是把每个 view/ranker 的 hot groups 当作 ranked localization results, 用 hidden labels 计算 precision@k、recall@budget、F1、AUPRC-style AP、nDCG 和 work-to-first-positive。比较对象包括 flat、fixed-session、dataset-native hierarchy、raw action stack、operation-stack、label-drilldown oracle view, 以及 width、visible-risk、query-aware 和 oracle-upper-bound rankers。非 oracle policies 只使用可见字段; label-drilldown 和 oracle-upper-bound 只作为上界。表 2 给出主要 policies 的 median 结果。

Operation-stack query-aware profiling 在 top-5 groups 中只检查 median 0.0937 operation fraction, 而 flat summary 必须检查 1.0。相对 fixed-session query-aware drilldown, operation-stack top-5 recall 在 5/6 tasks 上更高, 并把 median group count 从 285.0 降到 157.5。它并不在所有 accuracy 指标上支配其他 hierarchy。Dataset-native hierarchy 的 median top-5 recall 达到 0.8665, 但 work 高达 0.8539; fixed-session 的 work-to-first-positive 只有 0.0044, 但 top-5 recall 只有 0.0233。这正是 profiling paper 可以支持的 scoped claim: operation-stack 是一个 nondominated Pareto point, 它降低 flat summary 的检查 work, 并在多数任务上减少 fixed-session 的 group fragmentation, 但不替代所有 drilldown 视图。

R330 对 R320 的 task-level policy scores 做 10,000 次成对 bootstrap, 单位是 6 个真实标注 task family, 而不是单个 operation。相对 flat width, operation-stack query-aware 的 AP mean delta 为 0.1219, 95% CI 为 [0.0406, 0.2175]; top-5 inspected work mean delta 为 -0.8168, CI 为 [-0.9653, -0.6568]; 30% budget recall mean delta 为 0.4510, CI 为 [0.3450, 0.6143]; work-to-first-positive mean delta 为 -0.9303, CI 为 [-0.9855, -0.8650], 四项都是 6/6 tasks 同向。相对 fixed-session query-aware, operation-stack 的 top-5 recall mean delta 为 0.1801, CI 为 [0.0542, 0.3127], top-5 F1 mean delta 为 0.1702, CI 为 [0.0549, 0.2870], group count mean delta 为 -178.0, CI 为 [-340.0, -39.0]。相对 width-only operation-stack, query-aware ranking 的 AP mean delta 为 0.0932, CI 为 [0.0110, 0.2184], top-5 work mean delta 为 -0.3101, CI 为 [-0.4209, -0.1997]。同一个审计也保留反例: flat top-5 recall 由于检查全任务而更高, fixed-session 的 top-5 work、work-to-first-positive 和 AP 与 operation-stack 是混合 tradeoff, raw-action stack 的 mapping comparison 也不稳定。因此 R330 支持的是 task-family 层面的方向稳健性, 不是 per-operation 独立性、human utility 或单一层级支配。

R331 进一步检查 R320/R330 的 ranking signal 是否

只是正例比例、group 大小或排序长度造成。它固定每个 visible policy 的 group 和 ranking order, 只把同一任务的 hidden positive operations 随机分配到同样大小的 group positions, 做 2,000 次 label-permutation null。Operation-stack query-aware 的 AP 在 6/6 tasks 上超过 95% null, median observed-minus-null AP 为 0.0759; 30% budget recall 在 5/6 tasks 上超过 null, median delta 为 0.0904。这说明主 AP/预算召回信号不能被 prevalence 和 group size 单独解释。但 R331 也保留边界: top-5 precision 只有 3/6 tasks 超过 null, work-to-first-positive 为 0/6; width-only operation-stack AP 在 5/6 tasks 超过 null, fixed-session 和 raw-action AP 都在 6/6 tasks 超过 null。因此 R331 支持 mechanism-isolation 和 baseline-realism, 不支持所有 hot-group 指标、label-free ranker 或 operation-stack 单一支配。

R332 进一步把 R320 的 visible policies 做成 view/depth task-fit audit, 不重新 profiling, 也不读取 oracle policies 做选择。Best visible AP 分散在 operation-stack、fixed-session 和 dataset-native 上, 计数分别为 3/6、2/6 和 1/6。Best top-5 F1 更分散: raw-action stack 为 2/6, dataset-native、fixed-session、flat 和 operation-stack 各为 1/6。Operation-stack query-aware 是 3/6 tasks 的 best AP policy、3/6 tasks 的 best 30% budget-recall policy、1/6 tasks 的 best top-5 F1 policy 和 2/6 tasks 的 best work-to-first-positive policy。同时, operation-stack query-aware 相对 fixed-session query-aware 在 4/6 tasks 上减少 group fragmentation, median group ratio 为 0.5543。Leave-task source selection 不能自动解决 view 选择: AP 和 top-5 F1 的 selected-equals-best 都是 0/6; AP 的 median selected regret 为 0.0593, 而默认 operation-stack query-aware 的 median regret 为 0.0045。因此 R332 的结论不是引入第三个 profiler 对象, 而是把 view、stack fields、predicates 和 ranker 都作为同一 operation 上的 query-time 配置暴露出来; fixed-session、span tree 和 dataset-native hierarchy 保留为 baseline 或 drilldown 视图。

R333 把 R320 scorer 在同一 tracked operation JSONL 上重跑一次, 导出完整 top-k 和 operation-budget inspection curves; 它不下载、不同步、不创建、不重标数据集, 也不改变 hidden-label-after-ranking 规则。该审计生成 810 个 visible inspection points 和 252 条 task-policy curve rows。在 30% inspected-work 内, operation-stack query-aware 的 median recall 为 0.3900; flat width 为 0.0000, fixed-session query-aware 为 0.3559, dataset-native query-aware 为 0.3377, raw-action query-aware 为 0.3325。在 20% inspected-work 内, operation-stack query-aware 的 median recall 为 0.2763, fixed-session query-aware 为 0.2422; 30% budget 是更清晰的 budgeted-recall 证据点。相对 flat width, operation-stack query-aware 在 6/6 tasks 上 top-5 inspected work 更低, 并且在 30% budget 下有正召回而 flat 没有。相对 fixed-session query-aware, operation-stack query-aware 的 top-5 recall 更高为 5/6 tasks, groups 更少为 4/6 tasks, 但 work-to-first-positive 更低只有 2/6 tasks。因此 R333 支持 inspection-cost 与 fragmentation tradeoff, 不支持 operation-stack 在所有成本指标上支配 fixed-session。

R334 进一步把 “less fragmentation” 从 “less work” 中拆开审计。它读取 tracked R320 policy scores 和 R333 inspection curves, 不下载、不同步、不创建或重标数据集。相

对 fixed-session query-aware, operation-stack query-aware 在 4/6 tasks 上减少 total groups 和 positive groups, 在 5/6 tasks 上用更少 ranked groups 覆盖 50% positives, median groups-to-50% delta 为 -31.5。在同一 30% inspected-work budget 下, operation-stack query-aware 在 5/6 tasks 上比 fixed-session query-aware 检查更少 groups, median group delta 为 -54.0, 同时 median recall delta 为 0.0275。但 R334 也保留 fixed-session work 反例: operation-stack query-aware 的 work-to-50% recall 更低只有 1/6 tasks, top-5 work 更低只有 2/6 tasks, work-to-first-positive 更低也只有 2/6 tasks。因此本文可以 claim fixed-session fragmentation reduction 和 budgeted-recall tradeoff, 不能 claim operation-stack 在所有 work 指标上支配 span-tree drill-down。

5.3 E3: 机制隔离和 actionability

R361 将本节的 evidence contract 固定为: claim 是 profiler 通过 stack fields、mapping/tagging rules、ranking policies、profile specs 和 boundary-derived operation fields 暴露 actionable optimization knobs; oracle 是 hidden labels only after profiling, R358 还用 held-out OSWorld-Human boundary positives 评分; baseline 包括 default semantic-width specs、patched specs、visible feature rankers、transfer policies、learned-boundary stacks、fixed-session 和 semantic-width stacks; metric 包括 accepted patches、AP delta、top-5 lift delta、first-positive-work delta、group reduction 和 top-5 work counterpoint; counterpoint 是 OSWorld-Human 需要 boundary-derived fields, 因此这是 actionable mechanism evidence, not automatic boundary discovery、not automatic patch selection。RQ3 问的是: localization gain 来自哪些机制, profile-guided 的 stack fields、rank features、mappings 或 boundary-derived fields 能否在不把 hidden labels 泄漏进 profiler input 的前提下改善输出。

R320 首先把 localization 分数解释成任务级诊断, 而不是只给 policy 排名。SATraj unsafe 说明 environment、phase、action 和 risky/write-action ranking 对安全问题有效; looping 说明 repeat_signal 有用但需要 prevalence-aware ranking; side-effect 指向更深的 write/input mapping; AgentNet step-quality 需要 action-family 定位后再进入 fixed-session examples; OSWorld-Human 则需要 boundary-derived fields。因此 actionability 的第一层结论是: profiler 指出该调 stack field、mapping、ranker 还是 drilldown, 而不是给出唯一默认 view。

Rust 机制消融说明这些诊断来自可复现的 profiler 配置。R322/R323 证明 stack-text rank rules 和 rank mode 可以从 profile spec 复现, 但它们对 safety/side-effect 过粗。R324 把 operation-level rank features 放进 Rust: visible mapped operation tokens 在 folding 前被匹配和聚合, semantic stack 的 feature ranking 在 AP 上赢 5/6 tasks、top-5 lift 赢 4/6、first-positive work 赢 5/6, coarse depth 在 4/6 tasks 上改善 AP 并减少 groups。R325/R326 进一步说明哪些字段有用或有害: repeat_signal、write-action 和 status=success 在不同 family 关键, SATraj loop-like 或 OSWorld-Human input-phase features 会误导; global/equal feature banks 常能保留收益, R325-guided repair 在 2/3 misleading-feature cases 上改善 AP, 但这些 repair 使用离线评分反馈, 因此不是自动 learned ranker。

View、depth 和 budget 审计把机制结果放回 profiling

tradeoff. Best AP 和 best top-5 F1 分散在 operation-stack、fixed-session、dataset-native、raw-action 和 flat views; leave-task source selection 不能稳定选中 best view。但 operation-stack query-aware 在 6/6 tasks 上进入 Pareto frontier, 相对 flat 在 6/6 tasks 降低 top-5 work, 相对 fixed-session 在 5/6 tasks 提高 top-5 recall, 并在 5/6 tasks 用更少 groups 达到 50% positives。25% recall target 下它覆盖 6/6 tasks, median work 为 0.2000 而 flat 为 1.0000, median groups 为 16.0 而 fixed-session 为 50.0; 50% recall 和 first-positive work 仍保留 fixed-session、dataset-native 或 raw-action counterpoints。Sequence-scope 和 oracle-depth 审计进一步说明: 30% work budget 下 positive-session recall 为 0.4669, 高于 fixed-session 的 0.3230; 24 个 oracle-depth rows 上 budget-30 positive-unit recall median 为 0.4342, 并在 20/24 rows 提高 fixed-session unit recall、22/24 rows 用更少 groups 达到 50% positive units。Positive-run proxy units 是 episode-derived proxy units, 不是 human intent; positive-session-without-unit fixed-session depth-gap counterpoint 仍然保留, 所以这只是 depth-aware triage, 不是自动发现所有 latent subtask 或 intent boundary。R344 也保留多指标反例: 30 个 baseline-metric comparisons 支持 operation-stack query-aware, 16 个是明确 counterpoints, nDCG 场景下 flat 或 dataset-native 可能胜出。

Actionability portfolio 把这些比较变成 reviewer 可以检查的优化建议。R335/R341 显示 36/36 objective rows 都有具体 profile-configuration action, 但 27/36 best visible policies 不是默认 operation-stack query-aware; transfer miss 多数对应 view 或 ranker 变化, 而不是不可解释错误。R345/R346/R347 把同一证据组织成 diagnostic lenses、case packets 和 baseline contrasts: 6 个 lenses 覆盖 36 objectives, top-5 case groups 在 6/6 tasks 命中 positive, top-1 在 5/6 命中, median top-5 lift 为 1.6508 且 work 为 0.0937; operation-stack 在 6/6 tasks 胜过 flat top-5 work、在 5/6 胜过 fixed-session top-5 recall, 同时保留 fixed-session first-positive 和 flat full-recall 反例。R348/R349 则把 visible counterfactual 与目标标签泄漏分开: 36/36 best rows 是 visible non-oracle, 27/36 需要 non-default action, 25/36 需要 view change; held-out action transfer 在 35/60 aligned decisions 上 within tolerance, 但 exact action-class transfer 只有 7/60, 在 non-default target action 上只有 2/42。R350 最终把证据合成 bounded packets: 6/6 tasks top-5 含 positive, 5/6 top-1 含 positive, 4/6 满足严格 30% work budget, 同时保留 transfer guardrail。因此这里的 claim 是可调诊断 surface, 不是 label-free automatic selector。

R354/R358 关闭一个从诊断到可执行配置的闭环。R354 对同一 visible operation input 写 default 与 patched profile specs 并实际运行 Rust `agentpprof --profile-spec`, hidden labels 只在 profiler 输出后评分; 5/6 patches 同时改善 AP、top-5 lift 和 first-positive work, median AP delta 为 0.0376, median top-5 lift delta 为 0.5750。唯一 reject 是 OSWorld-Human, 说明 visible phase/action rank features 不足以处理 human-boundary 目标。R358 按这个诊断把 held-out boundary-derived fields 当作普通 operation fields 折叠, AP 从 semantic-width 的 0.2402 提升到 0.2583, groups 从 108 降到 74; 反例是 top-5 work 和 first-positive work 变差。R358 支持 boundary-derived fields 是 actionable operation-field knob, 不支持 automatic boundary discovery 或 automatic patch selector。

5.4 E4: 可复现性、成本和 claim integrity

R361 将本节的 evidence contract 固定为: claim 是 offline profiler path 和 paper claims 可复现、低成本且有 guardrails; oracle 是 tracked profile specs、tracked operation inputs、paper/doc hashes、claim-integrity audits 和 reviewer-style artifact checks; baseline 包括 deterministic versus raw replay、source-status checks、paper-number checks、rubric checks、artifact-review gates 和 two-abstraction guardrails; metric 包括 deterministic spec pass rate、runtime、number-check pass rate、guardrail pass rate、rubric level 和 artifact-review gate count; counterpoint 是 not live eBPF overhead、not human utility、not complete ecosystem compatibility、not universal selector。RQ4 问的是: reviewer 能否 replay offline profile-spec path, 并验证 source provenance、paper numbers、two-abstraction wording 和 non-claims 都保持一致。

R338 再把 paper claim 本身纳入审计: 它复用 R320–R350 的 tracked-clean artifacts, 不重新排名、不下载或重标数据集, 只检查论文数字、source policy、must-not-claim guardrails 和 operation/operation-stack 边界是否一致。因此它支持“论文表述与证据一致”这个提交前 gate, 而不是新的 profiler accuracy 结果。R352 再把当前结果集映射到 OSDI-style profiling-paper rubric: 它复用 tracked R320–R351 artifacts 和当前论文文本, 不下载、不同步、不重跑 profiler, 也不运行 human/agent analyst task。26/26 required checks 和 10/10 rubric areas 通过, 覆盖 fidelity/accuracy、actionability、baseline tradeoff、generality、mechanism isolation、robustness/statistics、reproducibility/overhead 和 claim-scope guardrails。因此它支持“当前 scoped profiling claim 的证据形态达到 profiler paper gate”, 不是新的 empirical result。R327 则检查 profile-spec 本身是否可复现、成本是否可报告: 所有 76 个 profile specs 在重复运行中保持 semantic profile content、samples 和 unique stacks 稳定, median 每个 spec 运行时间为 1.581s, p95 为 2.719s; 默认 JSON profile 的 raw-byte determinism 只有 4/76, 因为输出带生成时间戳。R328 再启用 deterministic artifact mode, 固定 JSON `generated_at` 和 pprof profile time; 同一 76 个 specs 的 semantic 与 raw-byte determinism 均为 76/76, clean rerun 记录空的 git/code status, median 每个 spec 运行时间为 1.601s, p95 为 2.767s。这些结果支持 artifact/reproducibility claim, 但不等价于 live capture overhead。这说明 profiler 输出不仅告诉我们哪里有问题, 还告诉我们哪类 stack depth、field mapping 和 rank policy 值得调。这些结论是 profiler 输出对优化方向的归因, 而不是人工 analyst 的主观解释。

R360 把表 5 的核心数字做成可复现 artifact: 脚本只读取 R360 source-status 中列出的 tracked upstream results 和当前 paper/docs, 输出 JSON、CSV、Markdown、HTML 和 LaTeX table fragment; 6/6 checks 通过。它不是新的 empirical result, 也不重新排名或同步数据集。R361 进一步把 E1–E4 表格整理成 claim-evidence ledger: 每个核心实验都必须给出 claim、research question、oracle、baselines、primary metrics、headline result、actionable insight、counterpoint、scoped wording 和 source runs。该 gate 10/10 checks 通过, 覆盖 hidden-label scale/metric surface、baseline tradeoff 而不是 metric dominance、oracle-depth/fragmentation、mechanism/actionability counterpoints、reproducibility/reviewer gates、two-

表 2: R320 hidden-label profiler benchmark. Hidden 表示 policy 是否读取 oracle label; WTFP 是 work-to-first-positive. 非 oracle policies 只读 visible operation fields, hidden labels 只用于 scoring。

Policy	Hidden	AP	nDCG	P@5	R@5	F1@5	Work@5	R@30%	WTFP
flat:width	no	0.1678	1.0000	0.1678	1.0000	0.2868	1.0000	0.0000	1.0000
fixed-session:query-aware	no	0.3476	0.7642	0.2555	0.0233	0.0427	0.0163	0.3559	0.0044
dataset-native:query-aware	no	0.3572	0.7445	0.1712	0.8665	0.2822	0.8539	0.3377	0.0582
raw-action:query-aware	no	0.2744	0.5012	0.2513	0.2442	0.2195	0.1507	0.3325	0.0374
operation-stack:width	no	0.1610	0.9364	0.1398	0.4746	0.2147	0.5424	0.3372	0.1694
operation-stack:query-aware	no	0.3117	0.5487	0.1991	0.1880	0.1981	0.0937	0.3900	0.0378
operation-stack:oracle-upper	yes	0.5993	0.6372	0.8984	0.3004	0.4029	0.0441	0.5640	0.0122
label-drilldown:oracle-upper	yes	1.0000	1.0000	1.0000	0.6703	0.7972	0.1150	1.0000	0.0377

表 3: R327 profile-spec 可复现性与离线执行成本。Runtime 为 Rust binary 已存在后的每个 spec 秒数; semantic hash 去除 JSON generated_at 字段, raw-byte hash 单独报告。

Workload	Specs	Med. s	P95 s	Med. stacks
R300 views	4	3.448	4.273	631.0
R324 rank	12	1.539	2.692	41.0
R326 robust	60	1.542	2.640	41.0
All specs	76	1.581	2.719	42.0

表 4: R327/R328 在同一 76 个 profile specs 上的 determinism 检查。R328 使用 --deterministic-output。

Run	Mode	Sem.	Raw	Med. s	P95 s
R327	default	76/76	4/76	1.581	2.719
R328	deterministic	76/76	76/76	1.601	2.767

abstraction boundary 和 must-not-claim guardrails。R361 仍然只是 paper-structure artifact, 不是新的实验, 不重新跑 profiler, 也不做 dataset sync、creation 或 re-labeling。R363 再把这些证据组织成五个 paper views: baseline-tradeoff scatter、metric heatmap、diagnostic lenses、actionability knobs 和 oracle-depth adequacy。该 portfolio 7/7 checks 通过, 作用是把 E2/E3 的 tradeoff 和 actionability 用多种分析视图展示出来, 而不是新增第五个实验、把论文降格为单一 flamegraph, 或引入 operation/operation-stack 之外的新抽象。

6 哪些数据集最适合

从当前 15 个方案看, 最适合支撑论文主 claim 的不是“最大”数据集, 而是同时具备顺序、动作、边界或质量 oracle 的数据集。第一类是 OSWorld-Human, 因为它同时给 single-action 和 grouped-action, 是验证递归边界的最直接 oracle。第二类是 AgentNet, 因为它规模大、人工桌面任务多, 并带 per-step correctness/redundancy。第三类是 AgentRewardBench 和 SATraj-OS, 因为它们把 profiler 从 action taxonomy 推到 failure、looping、安全和 side-effect diagnostics。第四类是 GUI-Odyssey/tau-bench/ToolBench 这组跨域补充, 它们覆盖移动 GUI、user-agent-tool dialogue 和 API/tool traces, 证明 operation abstraction 不局限于桌面。

ScaleCUA 是有用补充, 但不是当前主证据。R292 流式采样 5,000 个 Ubuntu navigation rows, 覆盖 131 个 sessions, 最大 step 为 48。该子集的动作 taxonomy 主要是 click 和 terminate, 所以 phase/action 指标达到 1.0 并不表示强化泛化。它更适合证明 multi-step GUI history context 可以被投影为 history_state 和 history_depth fields, 而不是证明复杂边界 detector。

7 相关工作

Profiling, flamegraphs 和 trace analysis. 传统 flamegraph 和 pprof 路线把运行时调用栈折叠成热点视图 [1,16,17]。pprof 不只是固定 call stack: 它支持 sample

tags, 也支持用 tagroot/tagleaf 把 tag value 作为 pseudo stack frame 可视化 [17]。Perfetto 则代表系统 trace ingestion、SQL trace analysis、trace summary 和 derived events 的成熟方向 [18]。因此本文不能把 novelty 写成“只有我们支持 query-time aggregation”。本文复用 folded-stack 表示, 但 scoped novelty 是 agent-operation record model、recursive multi-field operation-stack projection 和真实标注轨迹上的 hidden-label localization benchmark 这三者的组合。Stack frame 来自 agent operation fields, 而不是语言运行时函数调用或通用 trace SQL schema。因此同一批 operations 可以被折叠为 task、phase、tool、action、quality 或 safety 视图, 也可以改成 fixed-session 或 flat baseline。区别不在图形样式, 而在 agent 语义对象、递归多字段 stack projection 和 R320 的跨数据集 hidden-label scoring。

Agent tracing 和 LLM observability. OpenTelemetry GenAI 和 OpenInference 标准化 GenAI spans、LLM calls、agent reasoning steps、tool invocations、retrieval operations、MCP 和 provider-specific telemetry [2,11]。LangSmith、Langfuse、Phoenix 和 AgentOps 等系统进一步把 LLM call、tool call、latency、token、trace tree、session、observation、evaluation score、goal、plan 和 agent artifact 放进单次 workflow inspection 与生产监控 [12,13,14,15]。这些系统是本文最接近的同问题相关工作。因此本文把 trace/span/session 明确当作 exchange container 或 baseline shape, 而不是加成第三个 profiler 抽象。R320 实际评估的是 fixed-session drilldown 作为 span-tree proxy; 真实 OpenTelemetry/OpenInference/Phoenix span-tree import 仍是后续互操作 baseline, 不是当前已完成结果。本文关注跨轨迹 semantic profiling, 把异构标注轨迹或本地 agent 历史统一为 operations, 并允许用户选择 stack 深度、mappings、tagging 和 ranker。R314 专门检查这些 closest systems 是否进入 novelty map 和 baseline guard。

Agent failure 和 span localization. 2026 年的 AgentRx、TELBench/DRIFT、Holistic Evaluation and Failure Diagnosis 和 AgentAtlas 是最接近的同问题威胁 [23–26]。AgentRx 发布带 critical failure step 和 failure category 的失败轨迹, 并用 constraint checking 与 LLM judge 做 root-cause localization。TELBench/DRIFT 把 deep-research agent 轨迹转成 semantic spans, 并标注 harmful error spans。Holistic Diagnosis 把 agent-level diagnosis 和 span-level evaluation 结合起来; AgentAtlas 则强调 outcome leaderboard 不能覆盖 trajectory quality 和 control-decision quality。因此本文不能把 novelty 写成 failure localization、trajectory diagnosis 或 span-level evaluation 本身。本文的差异更窄: 把 profile groups 当作 profiler 输出, 在同一批 operations 上比较 flat、fixed-session proxy、dataset-native、raw-action 和 operation-stack views, 并用 AP、nDCG、inspection budget recall、work-to-first-positive、fragmentation 和 profile-configuration actionability 评分。AgentRx/TELBench-style traces 是后续 oracle/baseline 候

表 5: 四个 paper-facing 核心实验。R 编号只作为 provenance、ablation、counterpoint 或 audit gate 保留在 evaluation ledger; R360 从 tracked artifacts 重新生成该表的 headline metrics。

Core experiment	Workload / oracle	Main evidence	Scoped conclusion
E1: coverage, recursive folding, and field derivation	15 public labeled trace families / 47,590 operations, plus local-session and standard-trace exchange fixtures. OSWorld-Human and AgentNet provide the strongest boundary and step-quality oracles.	Core sources contain 42,590 operations, ScaleCUA brings the smoke set to 47,590, and the same R286 operations fold from 9 dataset stacks to 57 phase, 226 semantic, 455 action, or 3,757 fixed-session stacks. AgentNet profile-spec override changes 608 stacks to 83 without changing input operations. Held-out mapping improves compression from 14.049 to 19.091; OSWorld-Human learned boundary F1 reaches 0.7735 on held-out pairs.	Heterogeneous traces enter one operation layer, and stack depth is a recursive query over fields. Mapping, tagging, and boundary backends derive fields before folding; they do not add prompt/session/tool/safety-specific profiler objects.
E2: hidden-label localization and ranking	Six failure, safety, quality, and boundary tasks over AgentRewardBench, SATraj-OS, AgentNet, and OSWorld-Human; 34,539 operations / 3,699 positives.	R320 scores 144 view/ranker policies. Query-aware operation-stack top-5 groups inspect median 0.0937 work versus 1.0 for flat, improve top-5 recall over fixed-session on 5/6 tasks, and reduce median groups from 285.0 to 157.5. R333/R334/R337/R339/R355 add work-budget, fragmentation, fixed-recall, sequence-scope, and oracle-depth checks; R330/R331/R344 add uncertainty, permutation negative control, and multi-metric guardrails.	Operation-stack profiling is faithful to dataset-provided hidden labels under this benchmark and gives a better inspection-work/fragmentation Pareto point than flat or fixed-session alone. It is not a universal detector, a single best hierarchy, or a work-dominant replacement for drilldown views.
E3: mechanism and actionability	The same six labeled tasks, plus held-out OSWorld-Human boundary-backend operations from R297.	R324/R325/R326 show operation-level rank features, leave-one-feature ablations, and equal/global policies expose useful and misleading fields. R340/R341/R345-R350 show policy transfer, mechanism attribution, diagnostic lenses, casebook, baseline contrast, action counterfactuals, and evidence packets. R354 materializes 5/6 profile-spec patches; R358 shows learned-boundary fields improve OSWorld-Human AP to 0.2583 and reduce groups to 74, but increase top-5 work and first-positive work.	The profiler produces profile-configuration actionability: which stack fields, mappings, rankers, lenses, and drilldown depths to tune. R349/R354/R358 keep automatic action selection, automatic patch selection, and automatic boundary discovery out of scope.
E4: reproducibility, cost, and claim integrity	Tracked profile specs, tracked operation inputs, current paper/docs hashes, and no dataset sync or human/agent analyst task.	R327/R328 repeat 76 profile specs over tracked operation JSONL inputs. Semantic determinism is 76/76 in both modes; deterministic output makes raw-byte determinism 76/76 with median runtime 1.601s and p95 2.767s in the clean rerun. R319/R338/R352/R356/R357 audit implementation consistency, paper numbers, source provenance, rubric coverage, reviewer acceptance, and the two-abstraction boundary. R360 regenerates the 4-row core result table and 20 metric rows from tracked artifacts.	The artifact path is replayable and cheap enough for offline profiling-paper evidence. The current claim gates support scoped profiler-benchmark readiness, not live eBPF overhead, human productivity, full ecosystem compatibility, or universal selectors.

选，不是当前已完成的 broader span-localization 证据。

Computer-use 和 tool-use benchmarks. Weblinx、Mind2Web、GUI-Odyssey、OSWorld/OSWorld-Human、OpenCUA/AgentNet、SATraj-OS、AgentRewardBench、ToolBench 和 tau-bench 为本文提供真实标注轨迹 [3–10,19–22]。本文不提出新 benchmark。贡献是把这些 benchmark 的动作、质量、安全和人工 group 标签统一为 operation fields，并用同一个 profiler 机制比较它们适合回答哪些 profiling 问题。这些 benchmark 是 evidence source 和 native-sequence baseline，不是被本文替代的对象。

Novelty 边界。因此本文的新意不是“第一个 agent observability 系统”、不是“第一个 agent trajectory benchmark”、也不是“第一个 folded-stack 可视化”。当前可 defend 的 scoped novelty 是：只用 operation 和 operation stack 两个 profiler 抽象，把 trace/session/span 作为输入容器或 baseline，在真实公开标注轨迹上把 profile groups 当成 ranking/localization outputs，并用 hidden labels 计算 accuracy、work、fragmentation 和 actionability。更具体地说，新意不是 query-time aggregation 本身，而是把 agent-operation record、recursive multi-field stack shape 和 hidden-label localization benchmark 绑在一起：同一批 agent operations 因 stack shape 不同形成不同的可定位、可排序、可检查 aggregation units。这不是穷尽式 2026 全量综述；它是围绕当前 paper claim 的 closest-threat audit。

8 局限

当前结果仍不是 OSDI 或 NeurIPS 最终接收级完整证据。第一，field derivation 和 boundary recovery 的范围仍然有限。当前证据支持 deterministic mappings、label-derived rules 和部分 supervised boundary backends 能派生 stackable operation fields，但不证明无监督或 LLM-backed intent detector 已经解决，也不证明一个 backend 能跨所有标签家族工作。AgentRewardBench looping 被简

单 `repeat_signal_change` baseline 完全解释，这说明每个 family 都需要 suitability、calibration 和 simple-baseline gate。

第二，R320 支持 profiler-localization claim，但不支持人类效用 claim。R320 已经把 failure、safety、quality 和 human-boundary 问题转成 hidden-label ranking/localization benchmark，并证明 operation-stack profiling 对 flat summary 和 fixed-session drilldown 有 accuracy/work/fragmentation tradeoff。R332 进一步显示这个 tradeoff 是 task- 和 metric-specific 的：最佳 visible view 不由单一 hierarchy 垄断，source-task selector 也不能稳定自动选中最优 view。R333 进一步显示 20%/30% inspection budget 下 operation-stack query-aware 的 median recall 更高，但 fixed-session 仍保留 first-positive 低成本反例。R334 把 fixed-session fragmentation 与 operation-work 成本拆开：它支持更少 group fragmentation 和同预算更少 groups，但同时确认 first-positive/work 指标仍不由 operation-stack 支配。Related-work/baseline audit 只把 grounding 转成可失败检查，不增加新的实证结果。Protocol artifact 仍然有用，因为它把现有 visible packets 和 hidden answer key 固化为 6 个 tasks、3 个 views、24 个 participant slots 和 144 个 trials 的 balanced analyst-study protocol，并通过 visible-packet leakage check。但该 study 现在只是可选增强，用来验证开发者或 agent analyst 是否更快或更准。本文当前不能推出开发者准确率、time-to-answer 或 productivity 改善，也不能声称所有 intent boundary 都能被自动发现。论文主 claim 应固定为 profiler 本身在真实标注 trace 上的定位、排序、归因和 actionability，而不是 human utility。

第三，数据和互操作范围是轻量采样。本文避免同步完整测试集、图片归档和 gated 数据，因此有些来源只覆盖前缀、子配置或 redacted rows。Chrome Trace Event 和 agent-session exchange 当前是 exchange/reproducibility smoke；

R306 覆盖本地 session fixture, R353 覆盖 tracked real labeled operation prefix。它们证明 trace 可以作为 exchange container, 但不证明完整 OpenTelemetry/Chrome trace-ecosystem、Perfetto producer 或图像密集 benchmark 兼容。

第四, audit artifacts 是 claim-control surface, 不是额外实证结果。R338 进一步把 R320–R350 的论文数字、source policy、guardrails 和两抽象边界做成机械检查; result invariants、source-policy checks 和 guardrail checks 均通过。R352 再把这些底层结果映射到 profiling-paper rubric, 26/26 required checks 通过。这些 audit artifacts 提高了论文表述、source path、负结果、related-work grounding、evaluation-rubric coverage 和 must-not-claim 边界的一致性, 但它们只能审计底层 empirical artifacts。如果底层采样或 oracle 本身有偏差, audit 能暴露并收窄 claim, 不能消除偏差。因此本文最终主张应固定为 two-abstraction mechanism 和 hidden-label profiler fidelity/work trade-off, 而不是 universal fixed-session dominance、automatic anomaly detection 或完整 human utility。

9 结论

本文把 agent semantic profiling 收敛为两个核心抽象: operation 和 operation stack。Prompt、session、toolcall、process、syscall、plan、subagent、GUI action、安全标签和质量标签都只是 operation shapes 或 fields。Mapping 和 tagging 负责派生 fields, --view、--where 和 --stack 负责选择权重、查询子集和递归折叠形状, --trace-file 和 --operation-file 负责数据入口, --profile-spec 只把这些选择变成可复现实验配置, --deterministic-output 只把 profile artifact 时间戳固定为可复现值。在 15 个轻量采样的公开标注轨迹数据源上, 这个模型统一了 web、mobile、desktop、software、API、dialogue、failure、安全、human group 和 step-quality 轨迹, 并产生 flamegraph 之外的 stack tree、transition、quality、boundary、case packet 和 claim-audit reports。这支持本文的核心机制 claim: agent profiling 的边界可以从 prompt/session/span tree 中释放出来, 成为 query-time operation-stack projection。

当前最强经验结论来自四类 oracle-rich 轨迹。OSWorld-Human 证明同一动作序列可以折叠到人工 grouped-action 深度; AgentNet 证明 step correctness 和 redundancy 可以作为普通 operation fields 进入 profiling; AgentRewardBench 和 SATraj-OS 证明 failure、looping、side-effect、安全和 attack-type diagnostics 可以用同一机制表达。6 个 label-hidden tasks 构成主 hidden-label profiler benchmark: query-aware operation-stack top-5 groups 用 median 0.0937 work 取得 0.188 recall, flat summary 用 1.0 work 才取得 full recall, fixed-session 用 0.0163 work 但 top-5 recall 只有 0.0233; 相对 fixed-session, operation-stack top-5 recall 在 5/6 tasks 上更高, 并把 median groups 从 285.0 降到 157.5。

后续证据不是新的主实验, 而是解释这个 profiler claim 何时成立。Task-paired uncertainty、label-permutation negative controls、view/depth task-fit、inspection-budget curves、fixed-recall target、sequence-scope scoring 和 oracle-depth scoring 共同说明: operation-stack 的主要优势是 AP/work/fragmentation tradeoff, 而不是所有指标支配。Actionability 证据进一步显示 36/36 objective rows 有具体 optimization action, 27/36 best visible policies 不是

默认 operation-stack query-aware; diagnostic lenses、case packets、baseline contrasts、action counterfactuals 和 held-out transfer guardrails 说明 profiler 暴露的是可调分析 surface, 而不是 automatic universal selector。Executable profile-spec patch 进一步证明这种 profile-configuration actionability 可以落到 Rust profiler 配置上: 5/6 patches 改善 AP、top-5 lift 和 first-positive work; OSWorld-Human reject 再由 boundary-derived operation fields 改善 AP 和 groups, 但保留 work 反例。

因此本文的新意不在单个 flamegraph, 而在两个抽象形成的统一接口。Mapping、tagging、profile specs、agent-session trace、Chrome/Perfetto-style trace exchange、boundary backends 和 reviewer evidence packets 都只围绕 operation fields 与 operation-stack queries 工作。它们让用户和实验者能在同一批 operations 上选择不同 view、派生不同 fields、折叠不同深度, 并审计每个 claim 的证据来源。当前 value claim 是相对 dataset-provided hidden labels 的 profiler fidelity、label-scored ranking/localization、inspection-cost/fragmentation trade-off 和 profile-configuration actionability; 不是 detector、human utility、baseline dominance、automatic patch selector 或 ecosystem compatibility。下一步不应无差别增加数据集, 而应沿两个方向提高 claim 强度: 一是把 field-derivation backend 做到跨家族 calibration 并稳定胜过简单 sequence-derived baselines, 二是在已有或新增真实 trace 中补更强的 instruction-step、solution-path 或 human subtask oracle; 只有在要写 ecosystem baseline claim 时才导入真实 OTel/Phoenix/LangSmith/Langfuse/Perfetto span-tree trace。Controlled human 或 agent analyst study 可以作为后续增强, 但不再是本文主 profiling claim 的前置条件。

参考文献

- [1] Brendan Gregg. Flame Graphs. <https://www.brendangregg.com/flamegraphs.html>.
- [2] OpenTelemetry GenAI semantic conventions. <https://github.com/open-telemetry/semantic-conventions-genai>.
- [3] McGill-NLP WebLINX. <https://huggingface.co/datasets/McGill-NLP/WebLINX>.
- [4] OpenGVLab GUI-Odyssey. <https://huggingface.co/datasets/OpenGVLab/GUI-Odyssey>.
- [5] WukLab OSWorld-Human. <https://github.com/WukLab/osworld-human>.
- [6] xiangai AgentNet. <https://huggingface.co/datasets/xiangai/AgentNet>.
- [7] McGill-NLP AgentRewardBench. <https://huggingface.co/datasets/McGill-NLP/agent-reward-bench>.
- [8] AI45Research SATraj-OS. <https://huggingface.co/datasets/AI45Research/SATraj-OS>.
- [9] AgentSuite tau-bench trajectories. <https://huggingface.co/datasets/AgentSuite/tau-bench-trajectories>.
- [10] OpenGVLab ScaleCUA-Data. <https://huggingface.co/datasets/OpenGVLab/ScaleCUA-Data>.
- [11] OpenInference specification. <https://arize-ai.github.io/openinference/spec/>.
- [12] LangSmith Observability. <https://docs.langchain.com/langsmith/observability>.
- [13] Langfuse Observability. <https://langfuse.com/docs/observability/overview>.
- [14] Arize Phoenix. <https://arize.com/docs/phoenix>.
- [15] AgentOps: Enabling Observability of LLM Agents. <https://arxiv.org/html/2411.05285v2>.
- [16] Brendan Gregg. The Flame Graph. <https://queue.acm.org/detail.cfm?id=2927301>.
- [17] Google pprof documentation. <https://github.com/google/pprof/blob/main/doc/README.md>.
- [18] Perfetto Trace Processor documentation. <https://perfetto.dev/docs/analysis/trace-processor>.
- [19] AgentRewardBench: Evaluating Automatic Evaluations of Web Agent Trajectories. <https://arxiv.org/abs/2504.08942>.
- [20] OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. <https://arxiv.org/abs/2404.07972>.
- [21] OpenCUA: Open Foundations for Computer-Use Agents. <https://arxiv.org/abs/2508.09123>.
- [22] Safactory: A Scalable Agentic Infrastructure for Training Trustworthy Autonomous Intelligence. <https://arxiv.org/abs/2605.06230>.
- [23] AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. <https://arxiv.org/abs/2602.02475>.
- [24] Where Do Deep-Research Agents Go Wrong? Span-Level Error Localization in Agent Trajectories. <https://arxiv.org/abs/2606.02060>.
- [25] Holistic Evaluation and Failure Diagnosis of AI Agents. <https://arxiv.org/abs/2605.14865>.
- [26] AgentAtlas: Beyond Outcome Leaderboards for LLM Agents. <https://arxiv.org/abs/2605.20530>.